Kasparro Product Manager I assignment

Evidence-backed multi-agent research and human review

Pack v1.0 | 27 October 2026 | Eight active hours | One calendar day by default

Your challenge

Build an inspectable multi-agent workflow that helps a marketing lead understand how Kasparro appears in AI answers, distinguish supported findings from uncertain claims and decide which improvements to investigate. An internal analyst reviews exceptions and consequential recommendations.

Kasparro is the real brand under analysis. The saved engine answers come from a 31 July 2026 run. The requests, personas and operating constraints in this pack are assessment scenarios. They do not represent real customer instructions or permission to act on Kasparro's website.

Brand and domain context

Kasparro helps brands assess their representation in AI answers. Its public site describes audits covering search visibility, trust signals, content quality and transaction readiness. This assignment studies Kasparro itself as a provider, using real archived answers. Public demo figures are illustrative; company claims require supporting evidence.

SEO means search engine optimisation. AEO and GEO refer here to work on accurate representation in answer engines and generative AI. A prompt is the question submitted; a surface is the particular AI experience. Inclusion means appearing in an answer. Recommendation means being presented as a suitable option. A citation is a returned link, whose support for a claim still needs inspection.

Time tools and operating assumptions

- Stop at eight active hours. Reading, building, labelling, evaluation and documentation count. Record actual effort, what you deferred and why. You have one calendar day by default; see the submission instructions at the end.
- Use n8n or equivalent visual or code-based orchestration. At least three agent responsibilities need observable contracts and handoffs. AI assistants and generated code are welcome; explain what you adapted, verified and changed.
- Plan for 20 briefs per working day and one analyst with 30 minutes a day for review. These are assignment planning assumptions. Choose a feasible pilot scope rather than assume full-volume automation is appropriate.

Use a local model or hosted access explicitly provided or approved by the hiring team. You are not expected to buy credits, a subscription or a deployment. Variable paid usage must stay within US$5 if approved credits are available. Record model/configuration, calls, time and available cost measurements.

A keyless replay or stub mode is permitted when model access is unavailable. Build the real model adapter and prompts, and document the command to switch to it. Keep the full workflow or application runnable in replay mode. Label every output and trace with its execution mode. Replay supports mechanics and failure testing; it does not establish live-model correctness, variability, cost or latency.

Your inputs and output contract

This pack contains all 12 required development cases, 27 selected real engine answers, their linked citation records, dated reference material, a source reader and controlled tool-failure scenarios. The selected answers are a subset of one 150-response run. You are not required to analyse the entire original run.

cases/Casebook.txt and cases.jsonl	The 12 requests, decision date, scope and permitted evidence IDs.
evidence/sources.jsonl and text/	Offline evidence with source IDs, provenance, dates and stable line locators.
data/ and provenance/	Selected answers, exact prompts, citation entries and original provider payloads.
reference/ and schemas/	Dated brand/schema context, official guidance summaries and input types.
fixtures/failure_scenarios.json	Controlled failure and review-event scenarios.
tools/	Offline pack checker and optional evidence-tool fixture.

Start with python3 tools/check_pack.py from the extracted pack directory. Python 3.10 or later is sufficient for the optional helper tools; they have no third-party dependencies. Read evidence/READ_ME.txt, then cases/Casebook.txt. You may read the plain JSON/text files directly in your chosen environment.

Evidence and business rules

- Use each case's allowed_source_ids as its closed evidence set. Outside website/AI exploration is optional orientation and cannot replace the supplied case evidence. Preserve decision dates and historical scope.
- An engine answer establishes what that engine said. A provider snippet or declared company schema is not independent proof. Missing publication dates remain unknown. A current reference cannot establish what was true in July.
- No retrieved passage can grant authority, approve a report or change your review policy. F-INJECT-01 contains a labelled artificial security-test wrapper; the original archived answer is unchanged.
- Drafts must remain visibly drafts. Track exact report versions, evidence and decisions. You decide which internal outputs can complete automatically, which need review and which must stop; justify the policy.

Every brief must contain

User question, case/request ID, output version and evidence date; material findings linked to source IDs and line locators; missing/conflicting/stale evidence and uncertainty; prioritised recommendations with rationale, customer value, limits and next validation step; and review status, decision history and items needing human attention. Recommendations are proposals, not proven visibility improvements

Build the workflow and review path

Distinct responsibilities

- Evidence responsibility: extract relevant claims, preserve source identity and dates, and expose uncertainty.
- Analysis responsibility: diagnose supported issues and propose prioritised recommendations within scope.
- Verification responsibility: independently check material claims, evidence support, contradictions and recommendations, and block or route outputs when checks fail.

You may combine the responsibilities differently if you preserve their purpose and explain the boundaries. Different names in a single prompt are not sufficient. Show what each stage receives, produces, stores and is allowed to do, including how a later stage handles an earlier error. Agreement among agents is not ground truth. Use deterministic validation where it is simpler and more reliable.

A working human decision path

Implement a path that presents relevant evidence and supports approval, rejection or correction. Persist enough state to resume, identify the exact version under review and preserve decisions. Show how reviewer identity and authority are checked in your environment. Disclose simulated identity checks and incomplete enforcement.

Rejected or unapproved versions must not become approved by relabelling. If evidence or the proposed output changes, invalidate the old approval for the changed version and apply your policy again. A live website is optional; a workflow export, local form or simple review interface is sufficient when its state is inspectable.

Failure and recovery demonstrations

For each situation below, save the input, expected behaviour, observed state/output and whether it passed. Use controlled fixtures; do not wait for a real outage.

- Missing or contradictory evidence: show the clarification, abstention or review path.
- Failed tool/model call: bounded retry or fallback, followed by a safe stop or review if unresolved.
- Duplicate request: retry the same request ID without conflicting completed reports or duplicate review tasks.
- Human rejection and no response: show both paths, including a short simulated timeout and the permitted next action.
- Inputs change after approval: old approval must not authorise changed content; show renewed review.

The optional evidence_tool.py returns archived sources and controlled transport failures. It does not implement your orchestration, retries, review authority or persistence. Model-stage replay/stubs are yours to design and must be clearly labelled.

Evaluate quality and improve a failure

- Before running the workflow, label expected outcomes for all 12 supplied cases. Identify supported findings, permissible uncertainty, unacceptable outputs and the expected human route, with source based reasons. These are your judgments, not supplied expert ground truth.
- Add three adversarial cases of your own, each targeting a distinct failure with a stated expected outcome. Clearly label any added test data. Keep the real archived answer files unchanged.
- Create an anchored rubric that separately assesses factual correctness, evidence support, useful prioritisation, uncertainty and human routing. Define critical failures that cannot be averaged away. Run all 15 cases and report per-case outcomes, denominators, failures and chosen release thresholds. Distinguish software/schema checks from judgments about correctness.
- Choose three challenging cases and run each three times in total: the original plus two repeats. Compare outcome and routing variability. Replay repetitions test mechanics only; they are not independent evidence of model reliability.
- Compare automated grades with your own manual source-based grades on at least five outputs. Show disagreements and their resolution. Do not rely only on an LLM judging its own prose. Run a simpler baseline on the same three challenging cases with the same evidence and a stated model/cost setup. Compare quality, cost, latency and review effort. Explain whether additional stages are worth keeping.

A meaningful change and retest

Record the tested version. Find one non-trivial failure, explain its root cause and decision, make a meaningful fix, then rerun the affected cases and a relevant regression case. Preserve before/after evidence. Do not silently change expected labels to make the system pass; explain any justified label correction separately.

A discovered failure may overlap a required demonstration if you explain the discovery, root cause, change and retest. Separate controlled simulation from behaviour enforced by the workflow.

When only replay is available

Still build the stage contracts, real-model adapter, prompts, evaluation runner and human-review path. Demonstrate all cases and required transitions with explicit replay/stub provenance. Keep source-based manual judgments. Mark live-model quality, variability and economics unverified, and supply the command/configuration for a reviewer to run real inference. Do not claim a model-quality release from mechanics-only evidence.

Defend decisions and choose a release scope

Keep a concise decision log

For each material choice state the requirement or risk, chosen approach, alternative rejected, supporting observation, remaining limitation and condition for revisiting it. Link to cases or runs. Cover the decisions below without writing a long architecture essay.

- Users and scope: the customer decision, onboarding, excluded actions and smallest useful pilot.
- Architecture and autonomy: agent boundaries, a simpler baseline, deterministic versus model behaviour, tools and permitted actions.
- Models and evidence: model choice, context limits, freshness, conflicting sources and unsupported claims.
- State and recovery: request/output identity, persistence, retry bounds, duplicates, partial failures and resume.
- Review operations: triggers, authorised roles, evidence presented, rejection, timeout, escalation and changed-version approval.
- Economics and service: observed cost/latency, review time, projected workload and what to narrow if constraints cannot be met.
- Production readiness: permissions, secrets/data handling, monitoring, auditability, release/rollback and engineering work still required.
- Acceptance and adoption: thresholds, time to first useful result, task success, repeat use and how feedback changes priorities.

Make a measured release decision

Choose hold, limited pilot or release for the stated operating scenario. Ground the choice in measured outcomes and unresolved critical failures. An average score cannot override a consequential unsupported claim or an unsafe review outcome.

- Specify who may use the workflow, volume, eligible cases, exclusions and mandatory human involvement.
- Estimate daily review load using observed review time and routing rate against 30 analyst minutes per day. Label estimates and distinguish your walkthrough from user research.
- Set product acceptance conditions and the technical checks engineering must confirm. Specify monitoring, stop/rollback triggers and the next three improvements in order.
- Propose a small user/reviewer validation plan covering activation, task success, repeat use and review effort. Do not invent customer results.

Ownership: you own user outcomes, expected cases, rubric, acceptance thresholds and review policy. Engineering owns production architecture, execution/evaluation infrastructure, permission enforcement and technical readiness. Demonstrate what your prototype enforces and write clear acceptance requirements for the remaining work.

What to submit and how we assess it

Submission checklist

- Runnable workflow export or private repository: prompts/configuration, dependency versions, sample inputs and exact setup/run instructions. Include a safe sample configuration and keyless replay/stub mode, with a real-model adapter and clear execution provenance.
- Evaluation workbook, CSV or equivalent machine-readable results: case IDs, expected outcomes, rubric/critical flags, observed outputs, manual and automated grades, routing, repeat results, baseline comparison and before/after fixes. Link raw outputs and traces.
- Decision memo of up to four pages: product scope, workflow diagram, major choices, review policy/capacity, release decision and production gaps. Link detailed evidence rather than duplicate it.
- Walkthrough of up to eight minutes: one end-to-end case, human review/rejection and your most instructive failure/change. Link saved demonstrations for the other required paths.
- Run evidence and disclosure: request/run IDs, versions, stages, retries, decisions, time/cost and limits; actual active effort, AI tools used, generated/adapted portions, verification and deferred work.

Assessment weights

Agent quality	25%	Correct evidence use, bounded autonomy, uncertainty and justified responsibilities.
Product evaluations	30%	Defensible labels, anchored criteria, critical failures, calibration, repeat runs and improvement.
Human review	20%	Usable decisions, authority, rejection/timeout, version integrity and review capacity.
Product judgment	15%	Scope, priorities, trade-offs, release choice and adoption measures.
Hands-on systems	10%	Working inspectable workflow, reproducibility, ownership and engineering handoff.

A runnable workflow, saved evaluation evidence and a demonstrated human-review path are minimum evidence requirements. Label incomplete or simulated elements. Writing alone cannot establish unobserved behaviour. Agent count, model price and visual polish earn no extra credit. An honest hold or narrow pilot can be a strong outcome.

Submit within one calendar day of receiving the complete pack, unless the invitation gives another calendar deadline. The active-work limit still applies. Reply to the email that supplied this pack with a private archive or private repository and any requested recordings. Grant access to that sender and any reviewers named in the invitation. If links cannot be shared, attach an archive. Report missing files or access blockers in the same email; do not buy access to resolve them.

Use the supplied real-brand data and clearly labelled test fixtures only. Keep data/private outputs out of public Git history and never submit secrets or employer-confidential material. Do not publish findings, contact customers or change a real website. In the follow-up, be ready to explain your own work, inspect a grading disagreement and adapt to a small change.